

CP Notebook: Tricky Implementation Patterns

Python-style pseudocode for the algorithms I tend to forget mid-solve.

DP Sanity Checklist

Before coding, write:

```
# state = ?
# answer = ?
# base = ?
# transition = ?
# order = ?
```

Usual bugs: missing empty state (0, None, empty interval), memo check too late, wrong loop order, giant state causing MLE, `mask == full` vs `mask == n`.

Tree DP: Return All States

Use one postorder pass when parent needs multiple answers.

```
def dfs(u):
    if not u:
        return (0, 0)
        # empty subtree: (best if u is skipped, best if u is taken)

    l_skip, l_take = dfs(u.left)
    r_skip, r_take = dfs(u.right)

    take = u.val + l_skip + r_skip
    # taking u forces both children to be skipped
    skip = max(l_skip, l_take) + max(r_skip, r_take)
    # skipping u lets each child choose freely

    return (skip, take)
```

Rule: avoid separate `take(u)` / `skip(u)` recursion unless memoized.

LCA

Single query on binary tree:

```
def lca(u, p, q):
    if not u or u == p or u == q:
        return u
        # hit a target node, or fell past a leaf

    left = lca(u.left, p, q)
    right = lca(u.right, p, q)

    if left and right:
        return u
        # p and q found on different sides -> u is the split point
    return left if left else right
    # both live on whichever side came back non-None
```

Many tree queries: binary lifting.

```
up[0][v] = parent[v]          #  $2^0 = 1$  step up
```

```
up[k][v] = up[k - 1][up[k - 1][v]]
#  $2^k$  ancestor = two  $2^{k-1}$  jumps chained together
```

Lift deeper node first, then jump both from high power down.

Interval DP

Contiguous segment, choose split/pivot.

```
for length in range(2, n + 1):
    # solve smaller intervals before bigger ones
    for left in range(n - length + 1):
        right = left + length
        for mid in range(left + 1, right):
            # try every way to split (left, right)
            dp[left][right] = max(
                dp[left][right],
                dp[left][mid] + dp[mid][right] + cost(left, mid, right),
            )
```

Burst Balloons: add sentinel 1s. `dp[l][r]` = best inside open interval `(l, r)`. Pivot `m` is last popped.

Digit DP

State: `pos`, `tight`, `started`, `extra_state`.

from `functools` import `cache`

```
@cache
def dfs(pos, tight, started, state):
    if pos == len(digits):
        return valid(started, state)
        # placed every digit; score this choice

    limit = digits[pos] if tight else 9
    # tight -> can't exceed the true digit here
    total = 0
    for d in range(limit + 1):
        next_tight = tight and d == limit
        # still pinned to the prefix so far?
        next_started = started or d != 0
        # has a nonzero digit appeared yet?
        next_state = trans(state, d, next_started)
        total += dfs(pos + 1, next_tight, next_started, next_state)
    return total
```

If hand-memoizing, usually only memo when `tight == False`.

Bitmask DP

from `functools` import `cache`

```
full_mask = (1 << n) - 1
```

```
@cache
def dfs(mask):
    if mask == full_mask:
```

```

    return 0
    # everything is placed/visited already

best = INF
for i in range(n):
    if not (mask & (1 << i)):
        # bit i is unset -> i is still available
        best = min(best, cost(i, mask) + dfs(mask | (1 << i)))
return best

```

Checklist: | adds bit, & tests bit, base case reachable, memo active, loop indentation correct.

Shortest Paths

BFS: unweighted graph.

Dijkstra: nonnegative weights.

```

import heapq

dist = [INF] * n
dist[s] = 0
pq = [(0, s)]          # min-heap of (distance, node)

while pq:
    d, u = heapq.heappop(pq)
    if d != dist[u]:
        continue
    # stale entry: a shorter path to u was already popped
    for v, w in g[u]:
        if dist[v] > d + w:
            dist[v] = d + w
            heapq.heappush(pq, (dist[v], v))

```

Bellman-Ford: negative edges, negative cycle detection.

```

dist = [INF] * n
dist[s] = 0

for _ in range(n - 1):
    # a simple path has at most n-1 edges
    for u, v, w in edges:
        if dist[u] < INF and dist[v] > dist[u] + w:
            dist[v] = dist[u] + w

has_negative_cycle = False
for u, v, w in edges:
    # one more pass: any improvement now
    if dist[u] < INF and dist[v] > dist[u] + w:
        # means a negative cycle is reachable
        has_negative_cycle = True

```

Why $n-1$: shortest simple path has at most $n-1$ edges. One more improvement means reachable negative cycle.

Minimum Spanning Tree

Kruskal: sort edges + DSU. Better when edge list is natural/sparse.

```

parent = list(range(n))
size = [1] * n          # size of the tree rooted at each index

def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]
        # path halving: hop x toward its grandparent
    x = parent[x]
    return x

def union(a, b):
    root_a, root_b = find(a), find(b)
    if root_a == root_b:
        return False
    # already connected -> this edge would close a cycle

    if size[root_a] < size[root_b]:
        root_a, root_b = root_b, root_a
    # merge the smaller tree into the larger one

    parent[root_b] = root_a
    size[root_a] += size[root_b]
    return True

```

```

mst_weight = 0
for weight, u, v in sorted(edges):          # cheapest edges first
    if union(u, v):
        mst_weight += weight

```

Prim: grow from a node using cheapest crossing edge. Nice with adjacency list.

```

import heapq

visited = [False] * n
pq = [(0, 0)]          # (edge cost, node); start growing from node 0
mst_weight = 0

while pq:
    cost, u = heapq.heappop(pq)
    if visited[u]:
        continue
    # stale entry: u was already added via a cheaper edge
    visited[u] = True
    mst_weight += cost
    for v, edge_cost in g[u]:
        if not visited[v]:
            heapq.heappush(pq, (edge_cost, v))

```

If graph may be disconnected, count visited nodes or run Prim per component.

Toposort + DAG DP

```

from collections import deque

```

```

queue = deque(i for i in range(n) if indeg[i] == 0)
# start from nodes with no prerequisites
order = []

```

```

while queue:
    u = queue.popleft()
    order.append(u)
    for v in g[u]:
        indeg[v] -= 1
        if indeg[v] == 0:           # all of v's prerequisites are now done
            queue.append(v)

```

```

for u in order:
    # process in topological order so dp[u] is final
    for v, w in g[u]:               # before it's used to update dp[v]
        dp[v] = max(dp[v], dp[u] + w)

```

If `len(order) < n`, graph has a cycle.

Strongly Connected Components

Kosaraju:

```

def dfs1(u):
    seen[u] = True
    for v in g[u]:
        if not seen[v]:
            dfs1(v)
    order.append(u)
    # postorder: a node is appended only after all its descendants

```

```

def dfs2(u, c):
    comp[u] = c
    for v in rg[u]:           # walk the REVERSED graph this time
        if comp[v] == -1:
            dfs2(v, c)

```

```

seen = [False] * n
order = []
for i in range(n):
    if not seen[i]:
        dfs1(i)

```

```

comp = [-1] * n
c = 0
for u in reversed(order):    # highest finish time first
    if comp[u] == -1:
        dfs2(u, c)
        c += 1              # each dfs2 call floods exactly one SCC

```

SCC compression: build DAG with edges where `comp[u] != comp[v]`.

Max Flow

Ford-Fulkerson idea: keep finding augmenting paths in residual graph. BFS version = Edmonds-Karp.

from collections import deque

```

def bfs(s, t):
    parent = [-1] * n
    parent[s] = s           # mark s visited without a real predecessor

```

```

queue = deque([s])
while queue:
    u = queue.popleft()
    for v in range(n):
        if parent[v] == -1 and cap[u][v] > 0:
            # unvisited and has residual capacity
            parent[v] = u
            if v == t:
                return parent
            queue.append(v)
return None
# t is unreachable -> no augmenting path left

```

```

flow = 0
while True:
    parent = bfs(s, t)
    if not parent:
        break

    # walk the path backward to find its bottleneck (smallest residual capacity)
    push = INF
    v = t
    while v != s:
        u = parent[v]
        push = min(push, cap[u][v])
        v = u

    # apply that flow: forward edges shrink, reverse edges grow so it can be
    # undone later
    v = t
    while v != s:
        u = parent[v]
        cap[u][v] -= push
        cap[v][u] += push
        v = u

    flow += push

```

Reverse edges let you undo earlier flow choices.

Tree Diameter

Two BFS/DFS method for unweighted tree:

```

a, _ = farthest(0)
b, diameter = farthest(a)

```

Tree DP formula:

```

height[u] = 1 + max(height[child] for child in children[u])
# tallest subtree hanging off u
diameter = max(diameter, best_child_height_1 + best_child_height_2)
# longest path through u

```

Be consistent: heights in edges gives diameter in edges; heights in nodes gives diameter in nodes.

Range Query Trees

Fenwick: prefix sums / frequencies.

```
bit = [0] * (n + 1)          # 1-indexed internally

def add(i, x):
    i += 1                    # shift to 1-indexed
    while i <= n:
        bit[i] += x
        i += i & -i
        # move to the next index whose range covers this one

def sum_prefix(i):
    i += 1                    # shift to 1-indexed
    total = 0
    while i > 0:
        total += bit[i]
        i -= i & -i          # drop down to the parent range
    return total

def range_sum(left, right):
    return sum_prefix(right) - (sum_prefix(left - 1) if left else 0)
```

Segment tree: sum/min/max/gcd with point updates.

```
size = 1
while size < n:
    size *= 2
seg = [0] * (2 * size)
# seg[size + i] is leaf i; internal nodes hold subtree sums

def set_val(i, x):
    i += size                  # jump straight to the leaf
    seg[i] = x
    i //= 2
    while i:
        seg[i] = seg[2 * i] + seg[2 * i + 1]
        # recompute this ancestor from its two children
        i //= 2

def query(left, right):      # inclusive range [left, right]
    left += size
    right += size
    total = 0
    while left <= right:
        if left % 2 == 1:
            # left is a right child -> include it, then step past it
            total += seg[left]
            left += 1
        if right % 2 == 0:
            # right is a left child -> include it, then step past it
            total += seg[right]
            right -= 1
        left //= 2
        right //= 2
    return total
```

Use lazy propagation only for range updates.

Trie

Prefix tree for string sets / prefix queries.

```
class Trie:
    def __init__(self):
        self.children = {}    # char -> child Trie node
        self.end = False     # True if some inserted word ends exactly at this node

    def insert(self, word):
        node = self
        for c in word:
            # descend into child c, creating it on the fly if it doesn't exist yet
            node = node.children.setdefault(c, Trie())
        node.end = True       # mark the last node reached as a real word end

    def _walk(self, word):
        # follow `word` from the root; returns the landing node, or None if
        # the path breaks partway (some prefix of `word` was never inserted)
        node = self
        for c in word:
            if c not in node.children:
                return None
            node = node.children[c]
        return node

    def search(self, word):
        # exact word match: must reach the node AND it must be a word end,
        # not just a prefix of some longer inserted word
        node = self._walk(word)
        return node is not None and node.end

    def starts_with(self, prefix):
        # prefix match only: reaching the node is enough, end doesn't matter
        return self._walk(prefix) is not None
```

Every op is $O(\text{len}(\text{word}))$ regardless of how many words are stored — cost is the string length, not the dictionary size.

Array children (`[None] * 26`, index `ord(c) - ord('a')`) is faster when the alphabet is small and fixed.

Binary trie for max XOR: insert numbers as root-to-leaf bit paths (MSB first), then for each query greedily prefer the branch with the opposite bit — that maximizes the XOR at that position, and greedy is safe since a high bit outweighs all lower bits combined.

`BITS = 30` # highest bit index to consider; cover the max value in your input

```
def insert(root, x):
    node = root
    for b in range(BITS, -1, -1):      # MSB -> LSB, so numbers sharing a
        bit = (x >> b) & 1           # prefix share the same trie nodes
        if bit not in node:
            node[bit] = {}
        node = node[bit]

def max_xor(root, x):
```

```

# walk the trie trying, at every bit, to go the OPPOSITE way from x's bit
# (opposite bits XOR to 1); falls back to the same-bit child if the
# opposite branch doesn't exist for any number inserted so far
node, best = root, 0
for b in range(BITS, -1, -1):
    bit = (x >> b) & 1
    want = bit ^ 1
    if want in node:
        best |= 1 << b    # this bit differs -> contributes to the XOR
        node = node[want]
    else:
        node = node[bit]
return best

```

Pattern Map

Smell	Pattern
choose/skip on tree	tree DP states
contiguous segment	interval DP
count numbers $\leq N$	digit DP
subsets/visited set	bitmask DP
negative edges	Bellman-Ford
min network connection cost	MST: Kruskal/Prim
strongly connected directed groups	SCC
max bipartite/resource flow	max flow
prefix sums/frequencies	Fenwick
range min/max/sum + updates	segment tree
prefix lookup / autocomplete	trie
max XOR pair	binary trie